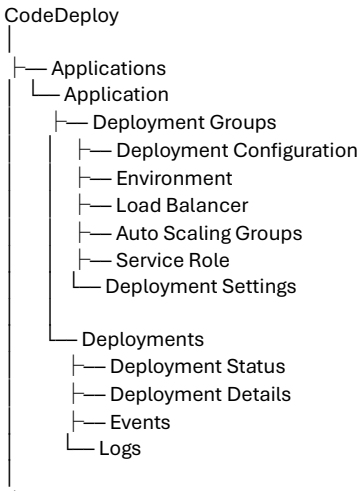
CodeDeploy — Console Dashboard Wise Important Interview Questions & Answers



1. What is AWS CodeDeploy?

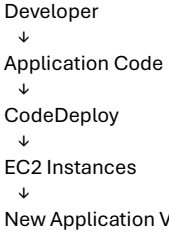
Answer:

AWS CodeDeploy is a deployment service used to automatically deploy application code to servers or compute services.
It can deploy applications to EC2 instances, on-premises servers, Lambda, and ECS.
It helps automate deployments and reduces manual deployment work.
We can also monitor the deployment status from the CodeDeploy console.

For example:

Suppose our application is running on multiple EC2 instances.
Instead of manually copying the new application files to every server, CodeDeploy can deploy the new version automatically.
We can monitor whether the deployment succeeded or failed from the console.

Example:



Simple ga gurthupettuko:

CodeDeploy = Automatically deploy application code

---

2. What is a CodeDeploy Application?

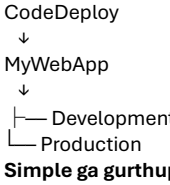
Answer:

A CodeDeploy Application is a logical container used to organize deployments for an application.
It stores the deployment configuration and deployment groups related to that application.
We create an application first and then create deployment groups under it.
The application itself does not contain the application code.

For example:

Suppose our application is called MyWebApp.
We create a CodeDeploy application named MyWebApp and then create deployment groups such as Production and Development.
Each deployment group can have different servers and deployment settings.

Example:



Simple ga gurthupettuko:

Application = Container for deployment configuration

---

3. What is a Deployment Group?

Answer:

A Deployment Group defines which servers or compute resources should receive a deployment.
It also contains important deployment settings such as deployment configuration, IAM service role, and load balancer settings.

For EC2 deployments, we can select instances using tags or Auto Scaling Groups.
Different environments can have separate deployment groups.
For example:
Suppose we have 10 production EC2 instances and 5 development instances.
We can create separate Production and Development deployment groups.
When we deploy to Production, CodeDeploy sends the application only to the production instances.
Example:
CodeDeploy Application
↓
Deployment Group
↓
Production EC2
EC2 EC2 EC2
Simple ga gurthupettuko:
Deployment Group = Where the application should be deployed

---

4. What is Deployment Configuration?
Answer:
Deployment Configuration defines how CodeDeploy should deploy the new application version to the target resources.
For EC2 deployments, it can control how many instances are taken offline or updated at a time.
Common strategies include AllAtOnce, HalfAtATime, and OneAtATime.
This helps control application availability during deployment.
For example:
Suppose we have 4 EC2 instances behind a load balancer.
With a suitable deployment configuration, CodeDeploy can update the instances gradually instead of updating all servers at the same time.
This helps keep the application available during deployment.
Example:
4 EC2 Instances
↓
CodeDeploy
↓
Update gradually
↓
Application remains available
Simple ga gurthupettuko:
Deployment Configuration = How deployment happens

---

5. What is the Environment section in a Deployment Group?
Answer:
The Environment section defines the resources where CodeDeploy should deploy the application.
For EC2 deployments, we can identify instances using tags or Auto Scaling Groups.
This allows CodeDeploy to automatically select the correct servers.
It is especially useful when the number of instances changes.
For example:
Suppose our production servers have the tag Environment=Production.
We configure the deployment group to use that tag, so CodeDeploy automatically targets those EC2 instances.
If Auto Scaling adds another matching instance, it can also become part of the deployment environment.
Example:
EC2
├— Environment=Production
├— Environment=Production
└— Environment=Development

CodeDeploy
↓
Production Tag
↓
Production EC2s
Simple ga gurthupettuko:
Environment = Which resources receive the deployment

---

6. What is the Load Balancer setting in CodeDeploy?
Answer:
The Load Balancer setting allows CodeDeploy to work with a load balancer during deployment.
CodeDeploy can remove instances from traffic while they are being updated and return them after deployment.
This helps prevent users from sending requests to an instance while its application is being replaced.
It is useful for highly available production applications.
For example:
Suppose four EC2 instances are behind an Application Load Balancer.

During deployment, CodeDeploy can take an instance out of traffic, update it, and then put it back into service.
It can repeat this process for the other instances.
Example:
ALB
↓
EC2-1 → Update → Back to Traffic
EC2-2 → Update → Back to Traffic
EC2-3 → Update → Back to Traffic
EC2-4 → Update → Back to Traffic
Simple ga gurthupettuko:
Load Balancer = Keep unhealthy/updating instances away from users

---

7. What is the Service Role in CodeDeploy?
Answer:
The CodeDeploy Service Role is an IAM role that allows CodeDeploy to perform required AWS actions on our behalf.
It gives CodeDeploy permissions to interact with resources required during deployment.
The role uses an IAM policy that defines these permissions.
Without the required permissions, CodeDeploy may not be able to perform the deployment correctly.
For example:
Suppose CodeDeploy needs to manage deployment-related actions for EC2 and Auto Scaling.
We provide a suitable IAM service role so CodeDeploy has permission to perform those actions.
CodeDeploy then assumes this role during the deployment process.
Example:
CodeDeploy
↓
IAM Service Role
↓
AWS Permissions
↓
Deployment Resources
Simple ga gurthupettuko:
Service Role = Permissions used by CodeDeploy

---

8. What is a Deployment in CodeDeploy?
Answer:
A Deployment is the actual process of installing a particular application version on the target resources.
It uses the application, deployment group, and deployment configuration to perform the deployment.
The CodeDeploy console shows whether the deployment is successful, failed, or in progress.
We can also open a deployment to see detailed events.
For example:
Suppose version v2 of our application is ready.
We start a deployment using the Production deployment group.
CodeDeploy installs v2 on the selected production servers and shows the deployment status.
Example:
Application v2
↓
CodeDeploy
↓
Production Deployment Group
↓
EC2 Instances
↓
Deployment Status
Simple ga gurthupettuko:
Deployment = Actual application update

---

9. What can we see inside Deployment Details?
Answer:
Deployment Details show the overall status and progress of a deployment.
We can see the target instances, deployment events, and whether individual steps succeeded or failed.
This helps us identify where a deployment problem occurred.
Logs can also be used for deeper troubleshooting.
For example:
Suppose a deployment fails on one EC2 instance.
We can open the deployment details and check the events to identify which deployment step failed.
Then we can check the related logs on the instance.
Example:
Deployment
↓
Deployment Details
↓

Events
↓
Failed Step
↓
Logs
↓
Troubleshoot
Simple ga gurthupettuko:
Deployment Details = Check deployment progress and failures

---

10. What are Deployment Events?
Answer:
Deployment Events show the individual steps performed by CodeDeploy during a deployment.
They help us understand what happened on each target instance.
If a deployment fails, events can indicate which lifecycle step caused the failure.
They are very useful for deployment troubleshooting.
For example:
Suppose CodeDeploy successfully downloads the application but fails while starting the application.
The deployment events can show the failed lifecycle step.
We can then check the application and CodeDeploy logs to find the actual issue.
Example:
Deployment
↓
Download
↓
Install
↓
Start Application ❌
↓
Check Logs
Simple ga gurthupettuko:
Events = Step-by-step deployment status

---

11. What are CodeDeploy Logs?
Answer:
CodeDeploy logs help us troubleshoot deployment problems on the target resources.
They can provide information about deployment lifecycle events and errors.
When a deployment fails, checking the logs is one of the important troubleshooting steps.
We can use the logs together with deployment events to identify the root cause.
For example:
Suppose an application deployment fails because a deployment script has an error.
We can check the CodeDeploy logs on the target EC2 instance to understand what went wrong.
After fixing the script, we can run the deployment again.
Example:
Deployment Failed
↓
Deployment Events
↓
CodeDeploy Logs
↓
Find Error
↓
Fix
↓
Redeploy
Simple ga gurthupettuko:
Logs = Find why deployment failed

---

12. What is a CodeDeploy Deployment Configuration vs Deployment Group?
Answer:
A Deployment Group decides **where** the application should be deployed.
A Deployment Configuration decides **how** the deployment should happen on those resources.
The deployment group contains the target resources and related settings.
The deployment configuration controls the deployment strategy.
For example:
Suppose we have 10 production EC2 instances.
The Deployment Group selects those 10 instances, while the Deployment Configuration decides how many instances should be updated at a time.
Both work together during deployment.
Example:
Deployment Group

↓
WHERE?
10 Production EC2s

Deployment Configuration
↓
HOW?
Update strategy
Simple ga gurthupettuko:
Deployment Group = WHERE
Deployment Configuration = HOW

---

13. What is the CodeDeploy Dashboard used for?
Answer:
The CodeDeploy dashboard provides an overview of applications and deployment activity.
It helps us quickly check deployment status and identify failed deployments.
From the console, we can move into applications, deployment groups, and individual deployments.
It is mainly useful for monitoring and managing deployments.
For example:
Suppose a production deployment is running.
We can open the CodeDeploy console and check whether the deployment is progressing successfully or has failed.
If it fails, we can open the deployment details and investigate the events and logs.
Example:
CodeDeploy Dashboard
↓
Applications
↓
Deployment Groups
↓
Deployments
↓
Status / Events / Logs
Simple ga gurthupettuko:
Dashboard = Monitor and manage deployments